

CS760 Midterm Exam

Instructions: Answer the following questions. Write out solutions below each question. Even if you cannot obtain a final answer, make sure to write your setup and explain how you would obtain the answer. Partial credit will be considered. Do not spend too much time on any single problem, as some problems are harder than others. Try to write only as much as needed to answer the question; the short answer questions that do not require derivations can typically be answered in 1-2 short sentences.

Wisc ID:

Name:

You get 3 bonus points for writing your name and Wisc ID correctly. (3 pts)

1 Evaluation

1.1 Metrics [5 points]

Describe one binary classification setting where accuracy might not be the best metric, and argue in favor of another metric instead.

Two example settings:

1. If most labels are negative, accuracy will not be very informative because a classifier that always outputs the negative label will do well. In this case it may be better to use a measure such as recall, which measures the proportion of the positive class that were correctly identified, or a combination of recall and precision in the form of the F_1 -score (since an always-positive classifier has good recall).
2. If a false negative might have a much higher cost than a false positive, then we may wish to understand how the model behaves at different cost differentials. A metric that captures this is the AUROC, which measures performance at different confidence thresholds on the labeling.

1.2 Validation [5 points]

Suppose I split my data into a training, validation, and test split, and I try to do empirical risk minimization with gradient descent. Which split do I use in each of the following scenarios, and why?

1. Tuning the number of iterations of gradient descent to obtain a better generalization error.
Tuning the number of iterations should be done using the **validation split**, so as to not leak information from the test set while still using an estimate of the how well the model generalizes.
2. I want to check if gradient descent is actually minimizing the empirical risk.
The empirical risk is defined over the **training split**.
3. I want to release my model along with an estimate of how it performs on unseen data.
The **test split** contains data unseen during model development and so can be used to estimate the performance on unseen data.

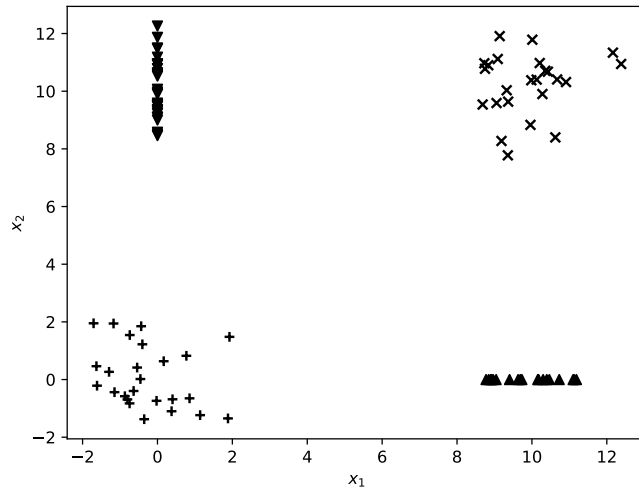


Figure 1: Data for Question 2 (and 3)

2 Supervised modeling

For each model class below, describe a simple instance that can perfectly classify the points in Figure 1 into “triangles” ($y = 1$) or “not triangles” ($y = 0$), or say briefly why it cannot be done. For the parametric models, you can use a hard threshold ($h(z) = 1_{z \geq 0}$) as the last layer rather than a sigmoid.

2.1 Decision tree [5 points]

- if $x_1 \geq 5$
 - if $x_2 \geq 5$ label $y = 0$
 - if $x_2 < 5$ label $y = 1$
- if $x_1 < 5$:
 - if $x_2 \geq 5$ label $y = 1$
 - if $x_2 < 5$ label $y = 0$

2.2 Linear classifier [5 points]

A linear classifier cannot perfectly classify these points because the classes are not linearly separable.

2.3 2-layer MLP with ReLU activation [5 points]

Let the hidden layer be $h_1 = \text{ReLU}(x_1 - x_2 - 5)$ and $h_2 = \text{ReLU}(x_2 - x_1 - 5)$. Output $y = 1$ if $h_1 + h_2 \geq 0$ and $y = 0$ otherwise.

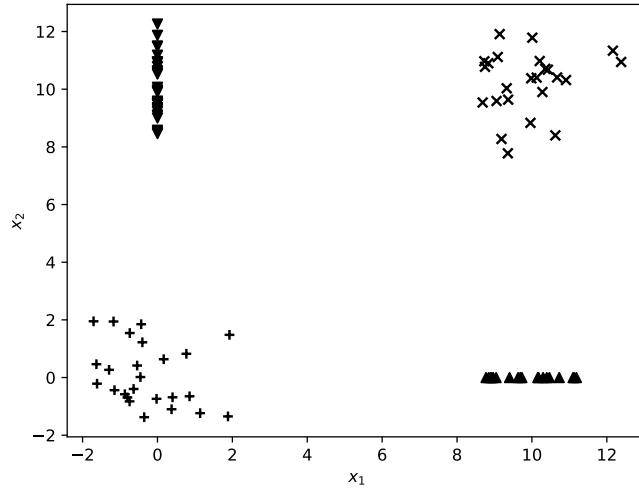


Figure 2: Data for Question 3 (and 2)

3 Unsupervised modeling

Below we refer to the points in Figure 2 (note this is the same figure as Figure 1).

3.1 Gaussian mixture model [5 points]

Suppose the data was generated by a Gaussian mixture model with four latent variables. Write down parameters $\phi_1, \dots, \phi_4 \in [0, 1]$, $\mu_1, \dots, \mu_4 \in \mathbb{R}^2$ and $\Sigma_1, \dots, \Sigma_4 \in \mathbb{R}^{2 \times 2}$ of a four-component Gaussian mixture model that could plausibly (in a probabilistic sense) generate the data. For simplicity, make the first latent variable (cluster) correspond to $+$, the second to x , the third to the up-pointing triangle ($\triangle$), and the fourth to the down-facing triangle. For simplicity, make the probability of each cluster be the same (so you do not need to count the number of points in each cluster) and only use the numbers 0, 0.25, 1, and 10.

1. $\phi_1 = 0.25, \mu_1 = \begin{pmatrix} 0 & 0 \end{pmatrix}, \Sigma_1 = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$
2. $\phi_2 = 0.25, \mu_2 = \begin{pmatrix} 10 & 10 \end{pmatrix}, \Sigma_2 = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$
3. $\phi_3 = 0.25, \mu_3 = \begin{pmatrix} 10 & 0 \end{pmatrix}, \Sigma_3 = \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix}$
4. $\phi_4 = 0.25, \mu_4 = \begin{pmatrix} 0 & 10 \end{pmatrix}, \Sigma_4 = \begin{pmatrix} 0 & 0 \\ 0 & 1 \end{pmatrix}$

3.2 Principal components analysis [5 points]

List one reason why principal components analysis (PCA) might not be useful to apply to this data.

Dimensionality reduction via something like PCA is often used for visualization, which is not applicable here as the data dimension is already two and so visualization is easy. PCA specifically can also be used to find a linear subspace that is close to most of the data points; in this case a (proper) linear subspace would be a line and the points do not lie close to any line.

4 Optimization

In this question, we will consider optimization of the following objective:

$$f(\mathbf{x}) = \frac{1}{n} \sum_{i=1}^n f_i(\mathbf{x})$$

We assume each $f_i : \mathbb{R}^d \mapsto \mathbb{R}$ is convex and has L -Lipschitz gradients, for known $L > 0$.

4.1 Gradient descent (GD) update [5 points]

Given we are at some point $\mathbf{x}_{t-1} \in \mathbb{R}^d$, write down the t th iteration of GD given a fixed step-size $\eta > 0$ and state how many gradient computations are required to compute it.

The GD update $\mathbf{x}_t \leftarrow \mathbf{x}_{t-1} - \frac{\eta}{n} \sum_{i=1}^n \nabla_{\mathbf{x}} f_i(\mathbf{x}_t)$ requires computing n gradients, one for each function f_i .

4.2 Number of iterations for gradient descent [5 points]

As we saw in class, if $\eta \leq 1/L$ then GD on f has suboptimality $f(\mathbf{x}_T) - f(\mathbf{x}^*) \leq \frac{\|\mathbf{x}_0 - \mathbf{x}^*\|_2^2}{2\eta T}$ after T iterations. Show that this implies the existence of a step-size that guarantees ε -suboptimality, i.e. reaching a point $\mathbf{x}_T \in \mathbb{R}^d$ s.t. $f(\mathbf{x}_T) - f(\mathbf{x}^*) \leq \varepsilon$, if we run gradient descent for at least $T = \frac{L\|\mathbf{x}_0 - \mathbf{x}^*\|_2^2}{2\varepsilon}$ iterations.

Using the formula, we require $\varepsilon \geq \frac{\|\mathbf{x}_0 - \mathbf{x}^*\|_2^2}{2\eta T}$; solving for T shows that ε -suboptimality holds for all $T \geq \frac{\|\mathbf{x}_0 - \mathbf{x}^*\|_2^2}{2\eta\varepsilon}$. This lower bound on T is minimized at the largest possible η for which the suboptimality bound holds, which is $\eta = 1/L$. Substituting that into the bound yields the result.

4.3 SGD update [5 points]

Given we are at some point $\mathbf{x}_{t-1} \in \mathbb{R}^d$, write down the t th iteration of SGD given a fixed step-size $\eta > 0$ and state how many gradient computations are required to compute it.

The SGD update samples i_t uniformly from the set $[n]$ and makes the update $\mathbf{x}_t \leftarrow \mathbf{x}_{t-1} - \eta \nabla_{\mathbf{x}} f_{i_t}(\mathbf{x}_t)$, which requires computing only one gradient, that of the function f_{i_t} .

4.4 Comparing bounds [5 points]

SGD on f is guaranteed to yield a point with (expected) suboptimality $\leq \varepsilon$ after $T = \left(\frac{2L\|\mathbf{x}_0 - \mathbf{x}^*\|_2^2 + C/L}{\varepsilon} \right)^2$ iterations, for constant $C \geq 0$ (Garrigos & Gower, Corollary 5.6). At least according to this upper bound and the one from Question 4.2, and ignoring expectations, show that we can have at most $n = \mathcal{O}(1/\varepsilon)$ functions f_i in the summation $f = \sum_{i=1}^n f_i$ for the gradient descent guarantee to be *better* than the SGD guarantee, in terms of the **number of gradient computations** sufficient to attain a fixed suboptimality $\varepsilon > 0$.

For any suboptimality $\varepsilon > 0$, the GD bound guarantees that $\frac{nL\|\mathbf{x}_0 - \mathbf{x}^*\|_2^2}{2\varepsilon}$ gradient computations suffice, whereas the SGD bound guarantees that $\left(\frac{2L\|\mathbf{x}_0 - \mathbf{x}^*\|_2^2 + C/L}{\varepsilon} \right)^2$ gradient computations suffice. Setting the former to be less than the latter and solving for n shows that it is worth running gradient descent if we have

$$n \leq \frac{(2L\|\mathbf{x}_0 - \mathbf{x}^*\|_2^2 + C/L)^2}{L\varepsilon\|\mathbf{x}_0 - \mathbf{x}^*\|_2^2} = \mathcal{O}(1/\varepsilon)$$

functions f_i in the summation.

4.5 When to use SGD instead of gradient descent [5 points]

If each i corresponds to a data point with loss f_i and minimizing $f = \sum_{i=1}^n f_i$ is thus doing empirical risk minimization, what does the previous bound suggest about the number of data points n and the required optimization precision (suboptimality) ε where it is often better to run SGD than gradient descent?

The bound suggests it is worth running SGD if we have a lot of data points (n is large) and/or are okay with a less precise solution (ε is large).

5 Regression

Suppose we model the data-generation process as $y = \mathbf{w}^\top \mathbf{x} + \epsilon$, where inputs $\mathbf{x} \in \mathbb{R}^d$ and an unknown parameter $\mathbf{w}$ are d -dimensional and $\epsilon \sim \mathcal{N}(0, \sigma^2)$ for some $\sigma \geq 0$. Recall that the p.d.f. of a normally distributed random variable $Y \sim \mathcal{N}(\mu, \sigma^2)$ is $f_Y(y) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y-\mu)^2}{2\sigma^2}\right)$.

5.1 Conditional probability [5 points]

Write down an expression for $P(y|\mathbf{x}, \mathbf{w})$.

$$P(y|\mathbf{x}, \mathbf{w}) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y-\mathbf{w}^\top \mathbf{x})^2}{2\sigma^2}\right)$$

5.2 Conditional log-likelihood [5 points]

Assume you are given a set of training observations $(\mathbf{x}^{(i)}, y^{(i)})$ for $i = 1, \dots, n$. Derive the conditional log-likelihood $\log P(\mathbf{y}|\mathbf{X}, \mathbf{w})$ of this training data, where $\mathbf{y} \in \mathbb{R}^n$ and $\mathbf{X} \in \mathbb{R}^{n \times d}$ aggregate the labels $y^{(i)} \in \mathbb{R}$ and inputs $\mathbf{x}^{(i)} \in \mathbb{R}^d$ into a vector and a matrix, respectively. Do NOT drop any constants.

The conditional probability is

$$P(\mathbf{y}|\mathbf{X}, \mathbf{w}) = \prod_{i=1}^n P(y^{(i)}|\mathbf{x}^{(i)}, \mathbf{w}) = \prod_{i=1}^n \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y^{(i)} - \mathbf{w}^\top \mathbf{x}^{(i)})^2}{2\sigma^2}\right)$$

and so the condition log-likelihood is

$$\begin{aligned} \log P(\mathbf{y}|\mathbf{X}, \mathbf{w}) &= \log \prod_{i=1}^n \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y^{(i)} - \mathbf{w}^\top \mathbf{x}^{(i)})^2}{2\sigma^2}\right) \\ &= \sum_{i=1}^n \log\left(\frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y^{(i)} - \mathbf{w}^\top \mathbf{x}^{(i)})^2}{2\sigma^2}\right)\right) \\ &= -\frac{n}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2} \sum_{i=1}^n (y^{(i)} - \mathbf{w}^\top \mathbf{x}^{(i)})^2 \end{aligned}$$

5.3 Equivalence to ordinary least-squares [5 points]

Argue briefly why maximizing the log-likelihood over $\mathbf{w}$ yields the same solution as minimizing $\|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2$.

The first term does not depend on $\mathbf{w}$, so we can drop it. Flipping the sign of the constant in front of the sum yields an equivalent minimization problem, and since the constant does not depend on $\mathbf{w}$ we can drop it. This yields $\arg \min_{\mathbf{w}} \sum_{i=1}^n (y^{(i)} - \mathbf{w}^\top \mathbf{x}^{(i)})^2$. Lastly, the quantity $\|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2$ is just the squared norm of a vector with entries $y^{(i)} - \mathbf{w}^\top \mathbf{x}^{(i)}$, making it equivalent to this summation.

5.4 Deriving the MLE [5 points]

Assuming $\mathbf{X}$ has full column rank, show that the MLE is $(\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$.

Note that $\|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2 = \mathbf{y}^\top \mathbf{y} - 2(\mathbf{X}\mathbf{w})^\top \mathbf{y} + \|\mathbf{X}\mathbf{w}\|_2^2$ and thus $\nabla_{\mathbf{w}} = -2\mathbf{X}^\top \mathbf{y} + 2\mathbf{X}^\top \mathbf{X}\mathbf{w}$. Taking the second derivative shows that the Hessian is $\mathbf{X}^\top \mathbf{X}$, which is p.s.d. since $\mathbf{X}$ has full column rank, making the objective convex. Therefore setting the gradient to zero and solving for $\mathbf{w}$ yields the global optimum.

5.5 Bayesian view of LASSO [5 points]

Assume a prior on $\mathbf{w}$ that each of its d entries is drawn i.i.d. from a Laplacian distribution $\text{Lap}(0, b)$. Write down the posterior probability $P(\mathbf{w}|\mathbf{y}, \mathbf{X})$ of $\mathbf{w}$, dropping scaling terms that do not depend on $\mathbf{w}$ (i.e. you can just derive what the posterior is proportional to). Recall that the p.d.f. of a Laplace-distributed random variable $Z \sim \text{Lap}(\mu, b)$ is $f_Z(z) = \frac{1}{2b} \exp\left(-\frac{|z-\mu|}{b}\right)$. Hint: use Bayes' rule, i.e. $P(A|B) = P(B|A)P(A)/P(B)$ for random variables A and B satisfying $P(B) \neq 0$.

The posterior probability is $P(\mathbf{w}|\mathbf{y}, \mathbf{X}) = P(\mathbf{y}|\mathbf{X}, \mathbf{w})P(\mathbf{w}|\mathbf{X})/P(\mathbf{y}|\mathbf{X})$. Since $\mathbf{w}$ is independent of $\mathbf{X}$ and $P(\mathbf{y}|\mathbf{X})$ does not depend on $\mathbf{w}$, we can write

$$P(\mathbf{w}|\mathbf{y}, \mathbf{X}) \propto P(\mathbf{y}|\mathbf{X}, \mathbf{w})P(\mathbf{w}) = \prod_{i=1}^n \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y^{(i)} - \mathbf{w}^\top \mathbf{x}^{(i)})^2}{2\sigma^2}\right) \prod_{i=1}^d \frac{1}{2b} \exp\left(-\frac{|\mathbf{w}_{[i]}|}{b}\right)$$

5.6 MAP estimation [5 points]

Show that the maximum *a posteriori* estimator of $\mathbf{w}$ with this prior is equivalent to minimizing the LASSO objective, which is $\frac{1}{2}\|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2 + \lambda\|\mathbf{w}\|_1$ for $\lambda = \frac{\sigma^2}{b}$.

Since the logarithm is monotonic we can maximize the log posterior probability, which decomposes into

$$\begin{aligned} \log P(\mathbf{w}|\mathbf{y}, \mathbf{X}) &\propto \log \prod_{i=1}^n \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y^{(i)} - \mathbf{w}^\top \mathbf{x}^{(i)})^2}{2\sigma^2}\right) + \log \prod_{i=1}^d \frac{1}{2b} \exp\left(-\frac{|\mathbf{w}_{[i]}|}{b}\right) \\ &= -\frac{n}{2} \log(2\pi\sigma^2) - \frac{\|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2}{2\sigma^2} - \log(2b) - \frac{1}{b} \sum_{i=1}^d \|\mathbf{w}_{[i]}\| \\ &\propto -\frac{\|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2}{2\sigma^2} - \frac{\|\mathbf{w}\|_1}{b} \end{aligned}$$

Maximizing the above is equivalent to minimizing the LASSO objective with $\lambda = \frac{\sigma^2}{b}$.

5.7 Effect of the prior on regularization strength [5 points]

How does varying the scale parameter b of the Laplace prior affect the regularization strength λ , and why does this effect make sense from the Bayesian view in which the entries of $\mathbf{w}$ being sampled from $\text{Lap}(0, b)$?

Decreasing b increases the regularization strength λ , which makes sense as a smaller scale parameter b means the entries $\mathbf{w}_{[i]} \sim \text{Lap}(0, b)$ of $\mathbf{w}$ are more likely to be close to the origin.

5.8 LASSO vs. Ridge regression [5 points]

State a property that an estimate of $\mathbf{w}$ computed via LASSO typically have that Ridge typically does not, and list a reason why this property can be useful.

LASSO typically yields sparse parameters $\mathbf{w}$, which can be useful when we want to recover a sparse signal from its (noised) linear transformation, when the data is high-dimensional ($d \gg n$), when we want to constrain the hypothesis class to sparse linear models, or for doing feature selection.